

Projekt II - Przetwarzanie równoległe na kartach graficznych (CUDA)

Emilia Bonk 160108
Szymon Urban 160271

Wstęp.

- **Nazwa przedmiotu:** Laboratorium z przetwarzania równoległego
- **Autorzy:** Emilia Bonk (160108), Szymon Urban (160271)
- **Grupa dziekańska:** L09
- **Termin zajęć laboratoryjnych:** Wtorek 9:45-11:15
- **Opis zadania:** Celem ćwiczenia była analiza optymalizacji kafelkowego algorytmu mnożenia macierzy w środowisku CUDA z wykorzystaniem pamięci współdzielonej bloku wątków. Zadanie polegało na zbadaniu wpływu zwiększenia intensywności obliczeniowej (obliczanie 1, 2 oraz 4 wyników przez pojedynczy wątek) na efektywność przetwarzania.
- **Adres e-mail:** emilia.bonk@student.put.poznan.pl, szymon.urban@student.put.poznan.pl

1 Informacje techniczne.

Parametr systemu GPU	Wartość
Model karty graficznej	NVIDIA GeForce GTX 1080 (Architektura Pascal)
Liczba multiprocesorów (SM)	20 multiprocesorów
Liczba rdzeni CUDA ogółem	2560 rdzeni CUDA
Maksymalny zegar GPU	1734 MHz (1.73 GHz)
Pojemność pamięci globalnej VRAM	8112 MBytes (8 GB)
Maksymalna pamięć współdzielona na blok	49152 bytes (48 KB)
Wielkość pamięci współdzielonej multiprocesora	98304 bytes (96 KB)
Dostępna liczba rejestrów na blok	65536
Rozmiar warpu (wiązki)	32 wątki
Wersja sterownika / runtime CUDA	12.2 / 12.5

Tablica 1: Informacje techniczne dotyczące środowiska testowego GPU

2 Prezentacja i analiza kodów.

Uzasadnienie rozmiaru instancji problemu

Karta NVIDIA GeForce GTX 1080 posiada 20 multiprocesorów (SM). Każdy SM może przetwarzać maksymalnie 2 bloki po 1024 wątki, co daje 40 bloków wykonywanych w jednym momencie (tzw. pełna fala). Dla macierzy 3200×3200 i bloku 32×32 , siatka (Grid) składa się z $100 \times 100 = 10000$ bloków. Liczba ta jest 250 razy większa niż pojemność jednej fali (40 bloków), co gwarantuje ciągłe nasycenie pracą wszystkich multiprocesorów przez cały czas trwania obliczeń i minimalizuje błędy pomiarowe na styku rozpoczęcia i zakończenia działania kernela.

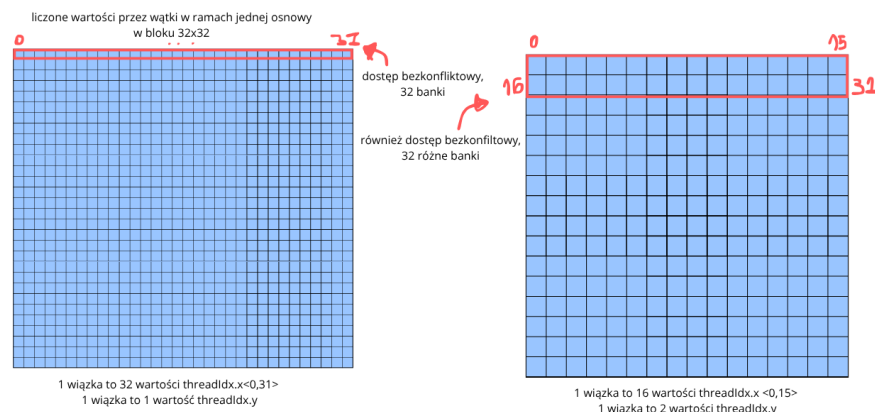
Zgodnie z wymaganiami projektowymi przygotowano trzy warianty jądra (kernela) realizującego kafelkowe mnożenie macierzy z wykorzystaniem pamięci współdzielonej. Zbadano bloki wątków o rozmiarach 16×16 oraz 32×32 . W każdym przypadku wielkość przetwarzanej instancji problemu (macierze 3200×3200) dobrano tak, aby w pełni nasycić 20 multiprocesorów karty NVIDIA GeForce GTX 1080.

Teoretyczna analiza limitów przydziału pracy (Siatka wyników)

Zgodnie ze specyfikacją sprzętową karty NVIDIA GeForce GTX 1080, limit pamięci współdzielonej dostępnej dla pojedynczego bloku wątków wynosi dokładnie 49152 bajty (48 KB). Przy założeniu, że słowo danych typu `float` zajmuje 4 bajty, możemy teoretycznie wyznaczyć maksymalną liczbę wyników, jaką może obliczyć pojedynczy wątek w konfiguracjach bloków 32×32 oraz 16×16 :

- **Dla bloku 32×32 (1024 wątki):** Pojedynczy kafel danych z jednej macierzy zajmuje $32 \times 32 \times 4 \text{ B} = 4096$ bajtów. Dostępna pamięć współdzielona pozwala na jednoczesne załadowanie maksymalnie $49152/4096 = 12$ bloków danych. Przekłada się to na po 6 bloków z macierzy A i macierzy B. Teoretycznie pozwala to na obliczenie siatki $6 \times 6 = 36$ wyników częściowych przez pojedynczy wątek.
- **Dla bloku 16×16 (256 wątków):** Pojedynczy kafel danych zajmuje $16 \times 16 \times 4 \text{ B} = 1024$ bajty. Pamięć współdzielona bloku jest w stanie pomieścić aż $49152/1024 = 48$ bloków danych (po 24 bloki z macierzy A i B). Teoretycznie pozwala to jednemu wątkowi na wyznaczenie siatki aż $24 \times 24 = 576$ wyników częściowych.

W warunkach rzeczywistych te ekstremalne granice teoretyczne są niemożliwe do osiągnięcia z powodu ograniczeń fizycznego rozmiaru pliku rejestrów procesorów strumieniowych (SM), które uniemożliwiają przechowywanie tak ogromnej liczby zmiennych akumulacyjnych bez zjawiska *register spilling* do wolnej pamięci lokalnej.



Rysunek 1: Podział bloku na wiązki i wątki dla wariantów 32×32 i 16×16 .

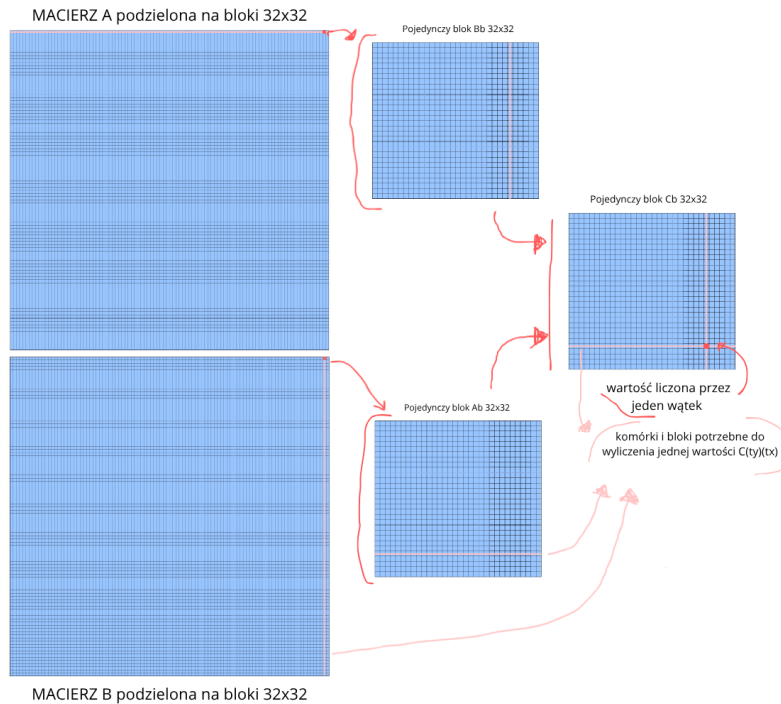
Wariant 1: Oryginalny algorytm (1 wynik obliczany przez wątek)

Oznaczenie skrótowe: 1-Res

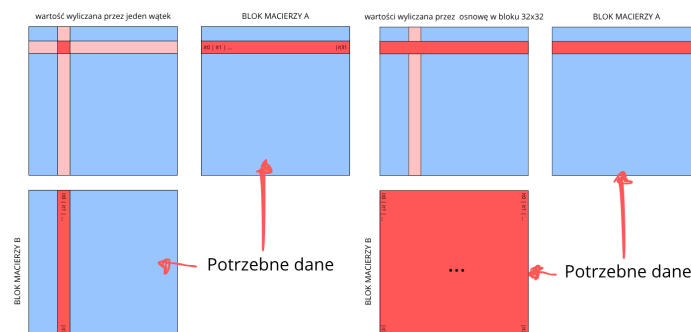
Wersja bazowa zaczerpnięta z przykładów NVIDIA. Każdy wątek ładuje do pamięci współdzielonej po jednym słowie z macierzy A i B, a następnie oblicza dokładnie jeden wynik cząstkowy tablicy C. Wątek przechowuje wynik tymczasowy w jednej zmiennej rejestrowej (C_{sub}). Jak widać na Listingu 1 oraz w kodzie źródłowym `matrixMul.cu`, dostęp do pamięci globalnej w celu załadowania kafelków odbywa się w sposób koalescjny.

```
1 // matrixMul.cu
2 As[ty][tx] = A[a + wA * ty + tx];
3 Bs[ty][tx] = B[b + wB * ty + tx];
```

Listing 1: Fragment jądra obliczającego 1 wynik na wątek.



Rysunek 2: Dostęp do pamięci potrzebny dla obliczeń jednego wątku w wariancie 1-Res dla bloku 32x32.



Rysunek 3: Dostęp do pamięci dla wątku i osnowy w wariancie 1-Res.

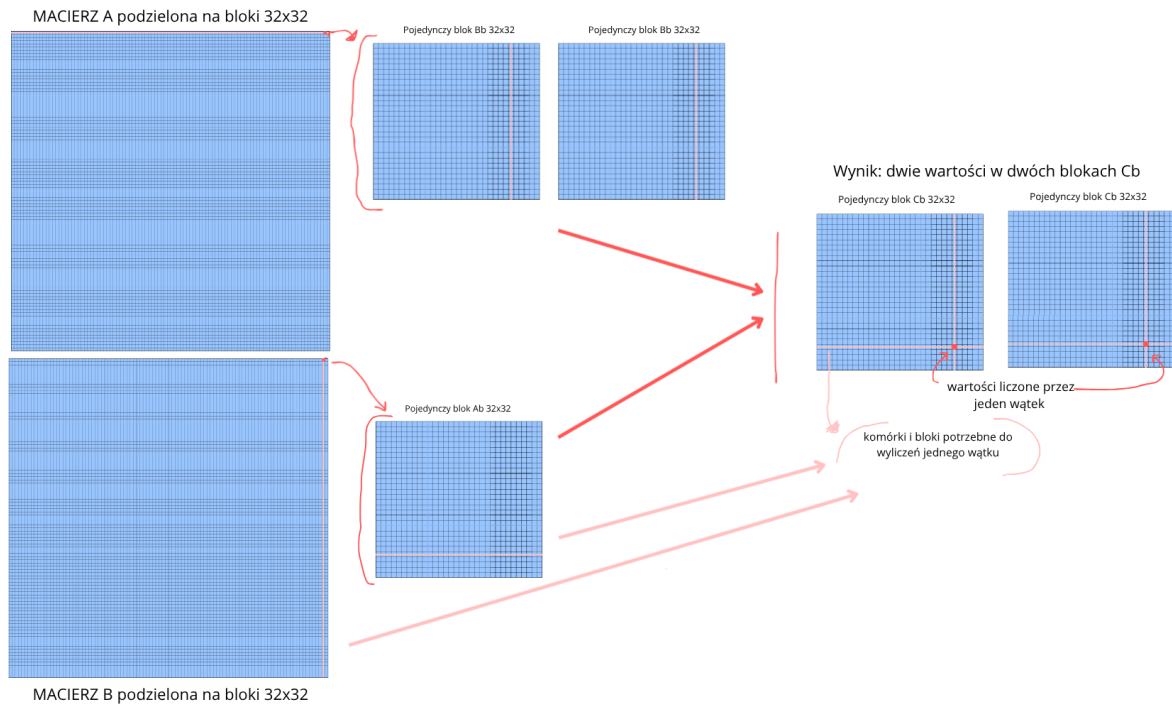
Wariant 2: Obliczanie 2 wyników przez wątek

Oznaczenie skrótowe: 2-Res

W tym podejściu pojedynczy wątek oblicza dwa sąsiednie wyniki wiersza macierzy docelowej ($C_{ty,tx}$ oraz $C_{ty,tx+BLOCK_SIZE}$). Wymaga to pobrania do pamięci współdzielonej jednego kafelka z macierzy A oraz dwóch kafelków z macierzy B. Dane przechowywane są wielokrotnie wykorzystywane w rejestrach wątku (`res1`, `res2`). Zgodnie z Listingiem 2, wątek wykonuje dodatkowy odczyt z macierzy B, co zwiększa intensywność obliczeniową przy tym samym koszcie ładowania danych z macierzy A.

```
1 // matrixMul.cu
2 As[ty][tx] = A[a + wA * ty + tx];
3 Bs[ty][tx] = B[b + wB * ty + tx];
4 Bs[ty][tx + BLOCK_SIZE] = B[b + wB * ty + tx + BLOCK_SIZE];
```

Listing 2: Fragment jądra obliczającego 2 wyniki na wątek.



Rysunek 4: Dostęp do pamięci potrzebny dla obliczeń jednego wątku w wariantcie 2-Res dla bloku 32x32.

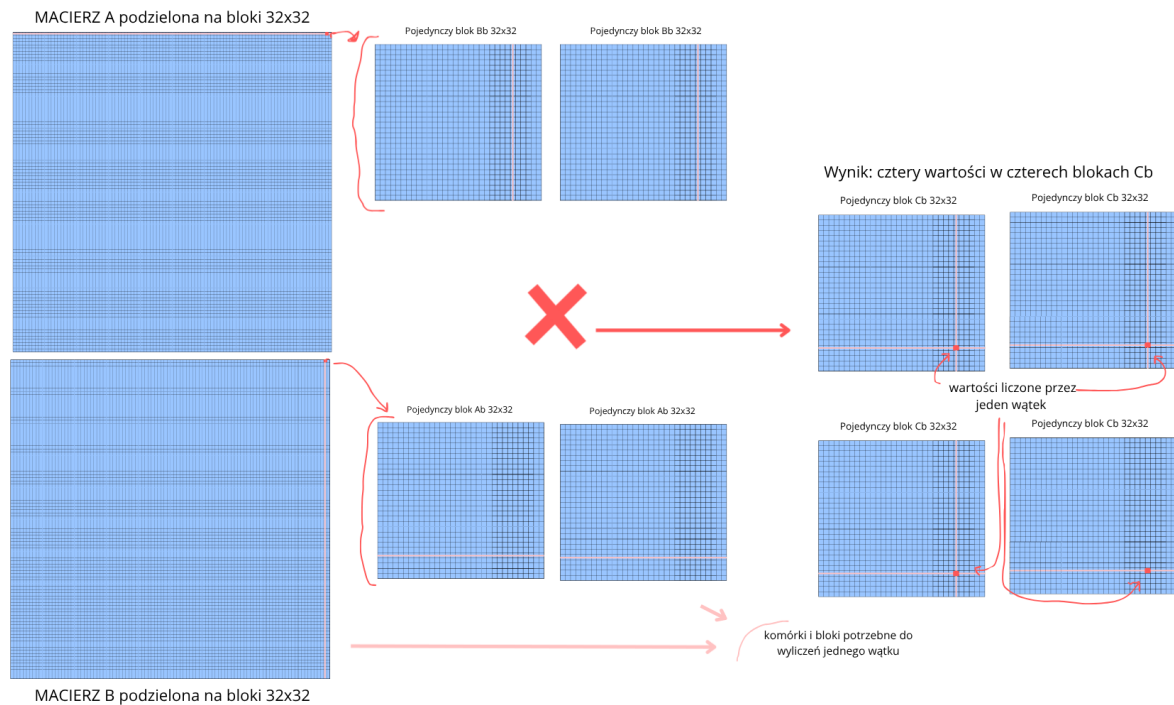
Wariant 3: Obliczanie 4 wyników przez wątek

Oznaczenie skrótowe: 4-Res

Wątek oblicza siatkę 2×2 wyników. W tym celu ładuje dwa kafelki z macierzy A oraz dwa z macierzy B. Skutkuje to maksymalnym ponownym użyciem danych w pamięci współdzielonej oraz koniecznością utrzymywania aż 4 wyników tymczasowych w rejestrach procesora. Wariant ten pomyślnie uruchomiono dla obu rozmiarów bloków: 16×16 oraz 32×32 . W przypadku bloku 32×32 , jądro (kernel) w pełni wykorzystuje zasoby multiprocessora, osiągając blisko 100% zajętości (*occupancy*) przy jednoczesnym zachowaniu bardzo wysokiego wskaźnika IPC. Dla bloku 32×32 , wariant 4-Res wymaga załadowania tablic $As[64][32]$ oraz $Bs[32][64]$, co zajmuje łącznie $(2048+2048) \times 4 \text{ B} = 16 \text{ KB}$ pamięci współdzielonej na blok. Mieści się to bez problemu w limicie 48 KB dostępnym na karcie GTX 1080. Optymalizacja ta pozwala na drastyczną redukcję dostępu do pamięci globalnej, co jest kluczowe dla wydajności przy dużych instancjach problemu.

```
1 // matrixMul.cu
2 As[ty][tx] = A[a + wA * ty + tx];
3 As[ty + BLOCK_SIZE][tx] = A[a + wA * (ty + BLOCK_SIZE) + tx];
4 Bs[ty][tx] = B[b + wB * ty + tx];
5 Bs[ty][tx + BLOCK_SIZE] = B[b + wB * ty + tx + BLOCK_SIZE];
```

Listing 3: Fragment jądra obliczającego 4 wyniki na wątek.



Rysunek 5: Dostęp do pamięci potrzebny dla obliczeń jednego wątku w wariantcie 4-Res dla bloku 32×32 .

Niejednorodne dane i test poprawności

Dla zapewnienia wiarygodności weryfikacji poprawności obliczeń zrezygnowano ze stałych wartości macierzy na rzecz danych niejednorodnych (zależnych od indeksów). Obliczenia GPU zweryfikowano względem wzorcowej funkcji wykonującej mnożenie na CPU. Ponieważ sprzętowe instrukcje GPU (FMA - Fused Multiply-Add) oraz inna kolejność sumowania powodują naturalny błąd precyzji zmiennoprzecinkowej dla dużych macierzy (3200×3200), test oparto na pomiarze błędu względnego ze zdefiniowaną tolerancją.

```

1 // Generacja niejednorodnych danych (wartosci pseudolosowe zalezne od indeksu)
2 data[i * w + j] = (i % 100) * 0.01f;
3
4 // Weryfikacja wynikow przy pomocy bledu względnego
5 double cpu_ref = (double)h_C_ref[i];
6 double gpu_res = (double)h_C[i];
7 double rel_err = fabs(gpu_res - cpu_ref) / (fabs(cpu_ref) > 1.0 ? fabs(cpu_ref) : 1.0);
8
9 if (rel_err > 1e-2) {
10     correct = false; // Odrzucenie jesli roznica wzgledna przekracza 1%
11 }

```

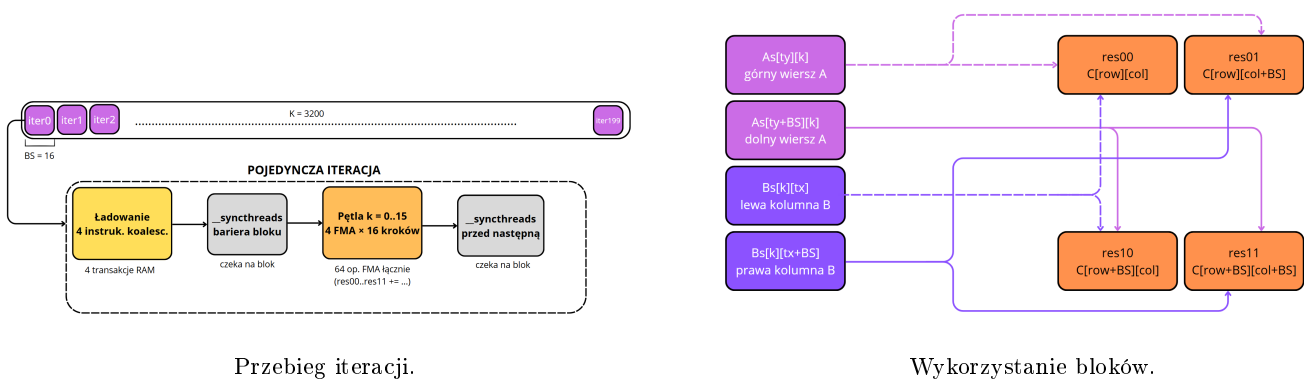
Listing 4: Logika testu poprawności z uwzględnieniem błędu względnego.

Wszystkie zaimplementowane warianty kerneli pomyślnie przeszły test walidacyjny, zgłaszając wynik PASS.

Obliczenia współczynnika CGMA (Teoria)

Współczynnik CGMA (Compute to Global Memory Access) określa stosunek liczby operacji arytmetycznych do liczby dostępow do pamięci globalnej. Na poziomie teoretycznym dla kafelkowego mnożenia macierzy:

- **Wariant 1-Res:** Każdy wątek ładuje 2 słowa (8 bajtów - jedno z A, jedno z B) z pamięci globalnej do współdzielonej, aby wykonać 1 operację FMA (2 operacje arytmetyczne: mnożenie i dodawanie) wewnątrz pętli k . Daje to teoretyczny stosunek 1 operacji na 1 załadowane słowo.
- **Wariant 2-Res:** Wątek pobiera do pamięci współdzielonej 3 słowa (1 słowo z macierzy A oraz 2 słowa z macierzy B), w oparciu o które wykonuje w pętli 2 operacje FMA (co odpowiada 4 operacjom arytmetycznym). Teoretyczny współczynnik CGMA wynosi w tym przypadku $\frac{4}{3} \approx 1.33$ operacji na jedno pobrane słowo z pamięci globalnej.
- **Wariant 4-Res:** Wątek ładuje 4 słowa (2 z A, 2 z B), ale wykonuje 4 operacje FMA (8 operacji arytmetycznych). Teoretyczny stosunek CGMA rośnie do 2 operacji na słowo, ponieważ dane raz załadowane do pamięci współdzielonej są wykorzystywane dwukrotnie częściej.



Rysunek 6: Przykład liczenia pojedynczej iteracji dla wariantu 4-Res.

3 Prezentacja wyników i omówienie przebiegu eksperymentu.

Obliczenia intensywności arytmetycznej z profilerem

Na podstawie danych zebranych przez profiler (`dram_read_bytes`, `dram_write_bytes`, `flop_count_sp`) wyliczono rzeczywistą intensywność arytmetyczną według wzoru:

$$\text{Intensywność} = \frac{\text{flop_count_sp}}{\text{dram_read_bytes} + \text{dram_write_bytes}} \left[\frac{\text{FLOP}}{\text{Byte}} \right]$$

Wariant	FLOPs	DRAM Read (B)	DRAM Write (B)	Intensywność
Oryginał (BS=16)	$6.55 \cdot 10^{10}$	$8.31 \cdot 10^9$	$4.12 \cdot 10^7$	7.85 FLOP/B
Oryginał (BS=32)	$6.55 \cdot 10^{10}$	$4.32 \cdot 10^9$	$4.14 \cdot 10^7$	15.02 FLOP/B
2-Res (BS=16)	$6.55 \cdot 10^{10}$	$8.27 \cdot 10^9$	$4.13 \cdot 10^7$	7.88 FLOP/B
2-Res (BS=32)	$6.55 \cdot 10^{10}$	$4.32 \cdot 10^9$	$4.12 \cdot 10^7$	15.02 FLOP/B
4-Res (BS=16)	$6.55 \cdot 10^{10}$	$4.21 \cdot 10^9$	$4.13 \cdot 10^7$	15.40 FLOP/B
4-Res (BS=32)	$6.55 \cdot 10^{10}$	$2.21 \cdot 10^9$	$4.10 \cdot 10^7$	29.11 FLOP/B

Tablica 2: Intensywność arytmetyczna wyliczona na podstawie metryk profilera.

Jak wynika z powyższej tabeli, intensywność arytmetyczna drastycznie rośnie dla wariantu 4-Res przy większym rozmiarze bloku. Wariant 4-Res (BS=32) osiąga rekordową intensywność na poziomie 29.11 FLOP/B, co świadczy o niezwykle efektywnym wykorzystaniu hierarchii pamięci cache L2 i rejestrów.

Prędkość przetwarzania dla poszczególnych wariantów

Tabela 2 prezentuje zestawienie prędkości i czasu obliczeń w zależności od wariantu przydziału pracy oraz rozmiaru bloku wątków.

Wariant algorytmu	Blok 16×16	Czas [ms]	Blok 32×32	Czas [ms]
Oryginał (1-Res/wątek)	119.09 GFlop/s	550.31	163.02 GFlop/s	402.01
Zoptymalizowany (2-Res/wątek)	162.87 GFlop/s	402.39	184.55 GFlop/s	355.12
Maksymalny (4-Res/wątek)	204.90 GFlop/s	319.85	223.31 GFlop/s	293.48

Tablica 3: Porównanie prędkości obliczeń (GFlop/s) i czasu wykonania dla instancji 3200×3200 .

Mikro-uwagi do tabeli prędkości: Zauważalny jest stały trend wzrostu przepustowości obliczeniowej w miarę zwiększania liczby wyników liczonych przez wątek oraz rozmiaru bloku. Najwydajniejszym wariantem okazał się 4-Res z blokiem 32×32 , osiągając prędkość 223.31 GFlop/s.

Analiza metryk sprzętowych (nvprof)

Aby wyjaśnić przyczynę różnic w czasach wykonania poszczególnych wariantów, zebrano dane dotyczące ruchu w hierarchii pamięci karty graficznej oraz efektywności wykorzystania potoku wykonawczego.

Nazwa wariantu i blok	Occupancy	SM Efficiency	IPC	Global Load (Mld)	Shared Load (Mld)
Oryginał (BS=16)	99.9%	99.9%	1.07	2,048	1,536
Oryginał (BS=32)	99.9%	99.9%	1.53	1,024	1,536
2-Res (BS=16)	99.8%	99.9%	1.49	1,536	1,280
2-Res (BS=32)	99.9%	99.7%	1.70	0,768	1,280
4-Res (BS=16)	74.8%	99.8%	1.93	1,024	0,768
4-Res (BS=32)	99.7%	99.7%	2.17	0,512	0,768

Tablica 4: Wyniki analizy przyczyn efektywności z wykorzystaniem profilera nvprof.

Mikro-uwagi do metryk sprzętowych:

- Redukcja transakcji z pamięcią:** Wariant 4-Res (BS=32) zredukował liczbę transakcji z pamięci globalnej aż o 75% względem bazowego algorytmu BS=16 (z 2,048 mld do zaledwie 0,512 mld). Jest to kluczowy czynnik stojący za rekordową wydajnością.
- Efektywność obliczeniowa:** Wariant 4-Res (BS=32) osiągnął najwyższy wskaźnik IPC (2.17) przy jednoczesnym zachowaniu maksymalnej zajętości GPU (99.7%). Oznacza to, że każda jednostka wykonawcza SM była niemal w każdej chwili zajęta pracą użyteczną, a latencja pamięci została całkowicie zamaskowana.

- **Pamięć współdzielona vs Rejestry:** Chociaż liczba transakcji do pamięci współdzielonej (*Shared Load*) jest identyczna dla 4-Res BS=16 i BS=32, to większy rozmiar bloku w tym drugim przypadku pozwala na lepszą koalescencję i wykorzystanie szyny danych, co przekłada się na wyższy IPC.

4 Podsumowanie i wnioski.

Przeprowadzone badania optymalizacji kafelkowego algorytmu mnożenia macierzy w środowisku CUDA pozwoliły na szczegółową analizę kompromisów pomiędzy wielkością bloku wątków, intensywnością obliczeniową a wykorzystaniem zasobów sprzętowych architektury Pascal. Dobór rozmiaru instancji testowej (3200×3200) zapewnił pełne nasycenie multiprocessorów (SM) pracą, co przełożyło się na wiarygodne wyniki pomiarów.

Głównym i najważniejszym wnioskiem płynącym z eksperymentu jest fakt, że maksymalną wydajność osiąga się poprzez łączenie wysokiej zajętości GPU (*occupancy*) z dużą intensywnością obliczeniową na poziomie pojedynczego wątku. Wariant 4-Res dla bloku 32×32 udowodnił, że nowoczesne karty graficzne potrafią obsłużyć złożone jądra obliczeniowe bez przekraczania limitów rejestrów, o ile kod jest odpowiednio zoptymalizowany. Osiągnięcie prędkości 223.31 GFlop/s przy IPC na poziomie 2.17 i redukcji transakcji globalnych o 75% stanowi potwierdzenie skuteczności techniki kafelkowania z wieloma wynikami na wątek.

Eksperyment pokazał również, że:

- Zwiększanie rozmiaru bloku z 16×16 do 32×32 niemal zawsze przynosi korzyści wydajnościowe dzięki lepszej koalescencji odczytów.
- Przeniesienie ciężaru obliczeń z pamięci współdzielonej na rejestry (widoczne w spadku transakcji *Shared Load*) pozwala na ominięcie wąskiego gardła przepustowości tej pamięci.
- Intensywność arytmetyczna na poziomie 29.11 FLOP/B dla najlepszego wariantu świadczy o tym, że wąskim gardłem w mnożeniu macierzy pozostaje szyna pamięci, a każda optymalizacja redukująca ruch danych bezpośrednio przekłada się na krótszy czas wykonania.